IND320 Project Part 3

November 10, 2026

1 IND320 Project Work [Part-3]

1.1 Data Quality

Student name: Emre Can Emer

GitHub Repository link: https://github.com/Emriey/IND320_project

StreamLit Share link: https://emriey-ind320-project.streamlit.app/

Screencast and .pdf files can be found in the GitHub repository.

1.2 AI Usage

Google Gemini was used for this assignment. I normally prefer to ask small questions if I feel like I'm stuck and look for guidance rather than direct solutions.

I do not have copilot enabled in VsCode.

In this assignment however, I did get help from Gemini with coding especially with signal processing. Even with scipy library, I needed help fixing certain errors or plots that weren't making any sense (converting to dB for the spectogram for example, which can be seen at the end).

1.3 Description

This jupyter notebook is used to fullfill the assignment requirements and also as testing grounds for the streamlit app.

List of fixes as requested by the teacher and suggested by the peer-review from the 2nd assignment: - Fixed the pie-chart on the charts page of the streamlit app. The entire page was redesigned and selection options were moved to the sidepanel. The pie chart shows correct amount of import and export values now. The plot on the right side column were also redesigned. It now resamples the data, using the mean of it, which helps with faster loading times and readability. It also responds correctly according to selected year.

- All the plots were changed to dark theme to fit the app better.

Log of main events: - Implemented requested fixes as explained above.

- Acquiring the shape files took a bit searching on my end. The website teacher showed in the lecture was just displaying the data. I had to find another NVE website to actually download it.

- STL and outlier plots look promising. However, I'm not sure about the precipitation outlier function. I feel like its baseline is too low and I couldn't get the parameters right for it.
- Temperature anomalies differ significantly depending on the season. Which made finding the right parameters a bit difficult.

Some disclaimers: - This project uses 100 randomly generated points instead of the requested 1000. The reason for this is because of API limitations. Even with batching and caching, I had problems fetching data for 1000 random points for each 'NO' zone. This also reduces load times significantly in the app. Please let me know if this is a strict requirement and if so, I'll try to fix it.

1.4 Initialization

[ ]:
```
# Importing the necessary libraries.
import numpy as np
import pandas as pd
import math
import time
import pickle

import matplotlib.pyplot as plt

import os
import sys
import requests

import geopandas as gpd
from shapely.geometry import Point
from scipy.fft import dct, idct
from sklearn.neighbors import LocalOutlierFactor
from statsmodels.tsa.seasonal import STL
from scipy.signal import spectrogram, detrend

import openmeteo_requests
import requests_cache
from retry_requests import retry

from dotenv import load_dotenv

# Setting up hadoop PATHs. I have them in windows environment paths too but␣
 ↪just in case.
# Spark is spark... you never know.
os.environ["HADOOP_HOME"] = "C:/Hadoop/hadoop-3.3.1"
os.environ["PYSPARK_HADOOP_VERSION"] = "without"
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
```

```
# I installed dotenv to hide my little secrets. This is one of the neat things␣
 ↪I learned from this assignment.
# All the sensitive info are stored in .env and secrets.toml files.
load_dotenv()
```

[ ]:
```
True
```

1.5 Downloading shape data

[2]:
```
# 1. Load the shapefile you downloaded from NVE (replace with your actual file␣
 ↪path)
# For electricity areas, it will be the file containing your bidding zones (e.g.
 ↪, budomraade.shp)
vaer_gdf = gpd.read_file("shape_files/Nedborfelt_RegineEnhet.shp",␣
 ↪encoding="latin1")
```

[3]:
```
vaer_gdf.head()
```

[3]:
```
        objType vassdragNr navnLokal  enhAreal  totAreal  \
0  REGINEenhet     001.21       NaN      0.12       0.0
1  REGINEenhet    001.224       NaN      5.96       0.0
2  REGINEenhet     001.33       NaN      4.19       0.0
3  REGINEenhet    002.114       NaN     21.17       0.0
4  REGINEenhet   002.1161       NaN      8.29       0.0

                                  hierarki  Q6190lskm2  regineQ  totTilsig  \
0             Haldenvassdraget/Iddefjorden       18.89     0.07        0.0
1             Haldenvassdraget/Iddefjorden       14.41     2.71        0.0
2             Haldenvassdraget/Iddefjorden       14.32     1.89        0.0
3  Glommavassdraget/Hvaler og Singlefjorden       10.79     7.20        0.0
4  Glommavassdraget/Hvaler og Singlefjorden       10.81     2.83        0.0

  vnrOFelt navnNedbf nbfHavNr navnNbfHav  niva  \
0      NaN  KYSTFELT      NaN   KYSTFELT  None
1      NaN  KYSTFELT      NaN   KYSTFELT  None
2      NaN  KYSTFELT      NaN   KYSTFELT  None
3      NaN  KYSTFELT      NaN   KYSTFELT  None
4      NaN  KYSTFELT      NaN   KYSTFELT  None

                                pktNavnFra  \
0    UTLÃ P AV ENNINGDALSELVA I IDDEFJORDEN
1       UTLÃ P AV KIRKEBEKKEN I IDDEFJORDEN
2   UTLÃ P AV UNNEBERGSBEKKEN I IDDEFJORDEN
3  UTLÃ P AV SKJEBERGBEKKEN I SKJEBERGKILEN
4                               MUNKETANGEN

                                    tilPktNavn  uttakDato  \
```

```
0                    UTLÃ P AV FOLKÃ
A I IDDEFJORDEN 2026-11-09
1  UTLÃ P AV TISTA (HALDENVASSDRAGET) I IDDEFJORDEN 2026-11-09
2                                          KJETANGEN 2026-11-09
3                                             BREVIK 2026-11-09
4                                       BEVÃ YTANGEN 2026-11-09

                     ekspType  \
0  NVEs nedlastningslÃ¸sning
1  NVEs nedlastningslÃ¸sning
2  NVEs nedlastningslÃ¸sning
3  NVEs nedlastningslÃ¸sning
4  NVEs nedlastningslÃ¸sning

                                                geometry
0  POLYGON Z ((297636.36 6543422.62 0, 297581.68 …
1  POLYGON Z ((293679.263 6558886.914 0, 293678.2…
2  POLYGON Z ((285561.85 6558497.64 0, 285636.23 …
3  POLYGON Z ((280690.8 6569492.91 0, 280732.53 6…
4  POLYGON Z ((275647.03 6563803.94 0, 275633.19 …
```

[ ]:
```
# Direct REST API URL for the Budområde layer (Layer 0).
# Taken from the website provided in the assignment page.
url = "https://kart.nve.no/enterprise/rest/services/Nettomraader/MapServer/0/
 ↪query?where=1%3D1&outFields=*&f=geojson"

# GeoPandas reads the URL and fetches the data.
electricity_gdf = gpd.read_file(url)
electricity_gdf = electricity_gdf.to_crs(epsg=25833)

print(electricity_gdf.head())
```

```
   objectid budomr globalid  st_area(shape)  st_perimeter(shape)  \
0         1   NO 1     None    6.052902e+10         1.657761e+06
1         2   NO 2     None    6.875969e+10         1.256473e+06
2         3   NO 3     None    1.042199e+11         1.773323e+06
3         4   NO 4     None    2.032500e+11         3.385499e+06
4         5   NO 5     None    3.334870e+10         1.106762e+06

                                              geometry
0  POLYGON ((337473.63 6970542.97, 337414.99 6970…
1  POLYGON ((-9833.91 6726499.86, -9861.7 6726719…
2  POLYGON ((308588.7 7261519.43, 295281.7 726804…
3  POLYGON ((973920.931 7962739.19, 973255.491 79…
4  POLYGON ((101466.43 6881318.54, 101382.41 6881…
```

1.6 Generating random points

[ ]:
```
def generate_random_points_in_polygon(polygon, num_points=100):
    """
    This function generates a specified number of random points strictly inside␣
 ↪a given polygon.
    """
    # Getting the bounding box (minx, miny, maxx, maxy) of the polygon.
    minx, miny, maxx, maxy = polygon.bounds

    points = []
    # Generate points until there is enough valid ones inside the polygon.
    while len(points) < num_points:
        # Generate a batch of random X and Y coordinates within the bounding␣
 ↪box.
        needed = num_points - len(points)
        x_coords = np.random.uniform(minx, maxx, needed * 2)
        y_coords = np.random.uniform(miny, maxy, needed * 2)

        # Need to check if the generated points are inside the polygons.
        for x, y in zip(x_coords, y_coords):
            p = Point(x, y)
            if polygon.contains(p):
                points.append(p)
                if len(points) == num_points:
                    break

    # Had problems with this. Needed to convert to match the coordinate system.
    return gpd.GeoSeries(points, crs="EPSG:25833")
```

[6]:
```
print(electricity_gdf[electricity_gdf['budomr'] == 'NO 3'][['budomr',␣
 ↪'geometry']])
```

```
  budomr                                           geometry
2   NO 3  POLYGON ((308588.7 7261519.43, 295281.7 726804…
```

[7]:
```
print(electricity_gdf['budomr'].unique())
```

```
<ArrowStringArray>
['NO 1', 'NO 2', 'NO 3', 'NO 4', 'NO 5']
Length: 5, dtype: str
```

[ ]:
```
# Decided write a pickle file to store the generated points so won't have to␣
 ↪generate them over and over.
# Dictionary to hold points for each zone.
zone_points_dict = {}

for zone_name in ["NO 1", "NO 2", "NO 3", "NO 4", "NO 5"]:
```

```
    # Filter your shapefile for the specific zone polygon
    zone_polygon = electricity_gdf[electricity_gdf['budomr'] == zone_name].
 ↪union_all()

    # Generating the 100 points. Had trouble with 1000 points because API␣
 ↪limitations.
    points_series = generate_random_points_in_polygon(zone_polygon,␣
 ↪num_points=100)

    # Convert to avoid coordinate errors.
    points_wgs84 = points_series.to_crs(epsg=4326)
    zone_points_dict[zone_name] = {
        "lats": points_wgs84.y.tolist(),
        "lons": points_wgs84.x.tolist()
    }

# Saving the points.
with open("zone_weather_points.pkl", "wb") as f:
    pickle.dump(zone_points_dict, f)
```

[ ]:
```
# Isolating NO 3 region as requested. Unlike the code above this part only␣
 ↪generates points for NO3.
no3_polygon = electricity_gdf[electricity_gdf['budomr'] == 'NO 3'].geometry.
 ↪union_all()

# Generating 100 points.
random_points_no3 = generate_random_points_in_polygon(no3_polygon, 100)

print(f"Generated {len(random_points_no3)} random points inside the region.")
```

```
Generated 100 random points inside the region.
```

[ ]:
```
def download_region_weather_batched(region_name, year, points_geoseries,␣
 ↪batch_size=10):
    """
    Downloads hourly ERA5 weather data.
    It is done in small batches because of API limitations.
    This code was taken the open-meteo and adjusted as needed.
    """
    # Caching is important here. No need to fetch the same data again.
    cache_session = requests_cache.CachedSession('.cache', expire_after=-1)
    retry_session = retry(cache_session, retries=5, backoff_factor=0.5)
    openmeteo = openmeteo_requests.Client(session=retry_session)

    # Converting again, as explained above.
    points_wgs84 = points_geoseries.to_crs(epsg=4326)
    lats = points_wgs84.y.tolist()
```

```
    lons = points_wgs84.x.tolist()

    url = "https://archive-api.open-meteo.com/v1/archive"
    all_locations_data = []

    # Dividing by the batches.
    total_batches = math.ceil(len(lats) / batch_size)
    print(f"Downloading data for '{region_name}' ({year}) across {len(lats)}␣
↪points in {total_batches} batches...")

    for i in range(0, len(lats), batch_size):
        batch_lats = lats[i:i + batch_size]
        batch_lons = lons[i:i + batch_size]

        params = {
            "latitude": batch_lats,
            "longitude": batch_lons,
            "start_date": f"{year}-01-01",
            "end_date": f"{year}-12-31",
            "hourly": [
                "temperature_2m",
                "precipitation",
                "wind_speed_10m",
                "surface_pressure"
            ],
            "models": "era5"
        }

        print(f"Fetching batch {i // batch_size + 1}/{total_batches}...")
        responses = openmeteo.weather_api(url, params=params)

        for j, response in enumerate(responses):
            hourly = response.Hourly()
            if hourly is None:
                continue

            global_idx = i + j
            hourly_data = {
                "location_id": global_idx,
                "latitude": response.Latitude(),
                "longitude": response.Longitude(),
                "date": pd.date_range(
                    start=pd.to_datetime(hourly.Time(), unit="s", utc=True),
                    end=pd.to_datetime(hourly.TimeEnd(), unit="s", utc=True),
                    freq=pd.Timedelta(seconds=hourly.Interval()),
                    inclusive="left"
                ),
```

```
                "temperature_2m": hourly.Variables(0).ValuesAsNumpy(),
                "precipitation": hourly.Variables(1).ValuesAsNumpy(),
                "wind_speed_10m": hourly.Variables(2).ValuesAsNumpy(),
                "surface_pressure": hourly.Variables(3).ValuesAsNumpy()
            }

            all_locations_data.append(pd.DataFrame(data=hourly_data))

        # Was having trouble with fetching data from the API. This seemed to␣
 ↪help.
        time.sleep(1)

    master_df = pd.concat(all_locations_data, ignore_index=True)
    return master_df
```

[ ]:
```
# Applying the function for NO3 for 2024
weather_no3_2024 = download_region_weather_batched("NO 3", 2024,␣
 ↪random_points_no3, batch_size=10)
print(weather_no3_2024.head())
```

```
Downloading data for 'NO 3' (2024) across 100 points in 10 batches…
Fetching batch 1/10…
Fetching batch 2/10…
Fetching batch 3/10…
Fetching batch 4/10…
Fetching batch 5/10…
Fetching batch 6/10…
Fetching batch 7/10…
Fetching batch 8/10…
Fetching batch 9/10…
Fetching batch 10/10…
   location_id  latitude  longitude                      date  temperature_2m  \
0            0      65.0      11.75 2024-01-01 00:00:00+00:00          -4.867
1            0      65.0      11.75 2024-01-01 01:00:00+00:00          -5.017
2            0      65.0      11.75 2024-01-01 02:00:00+00:00          -4.717
3            0      65.0      11.75 2024-01-01 03:00:00+00:00          -4.667
4            0      65.0      11.75 2024-01-01 04:00:00+00:00          -4.417

   precipitation  wind_speed_10m  surface_pressure
0            0.0       33.402489       1011.803894
1            0.0       33.586281       1011.503235
2            0.0       33.772911       1011.506714
3            0.0       33.962330       1011.706482
4            0.0       34.154495       1011.210876
```

1.7 Outliers

[ ]:
```
def analyze_precipitation_outliers(df_weather, cutoff_freq=5, num_stds=3):
    """
    This function analyzes regional precipitation for outliers using DCT␣
 ↪high-pass filtering
    and robust Statistical Process Control (SPC) boundaries.
    """
    # Aggregate to average precipitation across all points over time.
    # Making sure the timestamps are not str.
    df_sorted = df_weather.copy()
    df_sorted['date'] = pd.to_datetime(df_sorted['date'])

    # Grouping by time and calculate spatial mean for NO 3.
    time_series = df_sorted.groupby('date')['precipitation'].mean().
 ↪reset_index()
    time_series = time_series.sort_values('date').set_index('date')

    signal = time_series['precipitation'].values
    N = len(signal)

    # Applying Discrete Cosine Transform (DCT-II) for High-Pass Filtering.
    coeffs = dct(signal, type=2, norm='ortho')

    # Apply high-pass filter to zero out low frequencies.
    coeffs_filtered = coeffs.copy()
    coeffs_filtered[:cutoff_freq] = 0

    # Inverse DCT to get the Seasonally Adjusted Variations (SATV).
    satv = idct(coeffs_filtered, type=2, norm='ortho')

    # Setting the boundaries.
    median_satv = np.median(satv)
    # Median Absolute Deviation (MAD) scaled for standard deviation.
    mad = np.median(np.abs(satv - median_satv))
    robust_std = 1.4826 * mad

    upper_limit = median_satv + (num_stds * robust_std)
    lower_limit = median_satv - (num_stds * robust_std)

    # Identifying outliers based on SATV thresholds above.
    outlier_mask = (satv > upper_limit) | (satv < lower_limit)

    # Map back to original precipitation time series for summary/plotting
    time_series['SATV'] = satv
    time_series['Outlier'] = outlier_mask
```

```
    # Calculate SPC boundaries mapped to the original scale for visualization
    time_series['Upper_SPC'] = time_series['precipitation'] + (upper_limit - satv)
    time_series['Lower_SPC'] = time_series['precipitation'] + (lower_limit - satv)

    # Plotting the results.
    fig, ax = plt.subplots(figsize=(12, 6))

    # Plotting the baseline average precipitation curve.
    ax.plot(time_series.index, time_series['precipitation'], color='blue', alpha=0.6, label='Mean Precipitation')

    # Plotting the SPC boundaries.
    ax.plot(time_series.index, time_series['Upper_SPC'], color='orange', linestyle='--', label=f'+{num_stds} Robust SD Limit')
    ax.plot(time_series.index, time_series['Lower_SPC'], color='orange', linestyle='--', label=f'-{num_stds} Robust SD Limit')

    # Highlighting the outliers color.
    outliers = time_series[time_series['Outlier']]
    ax.scatter(outliers.index, outliers['precipitation'], color='red', zorder=5, label='Outliers')

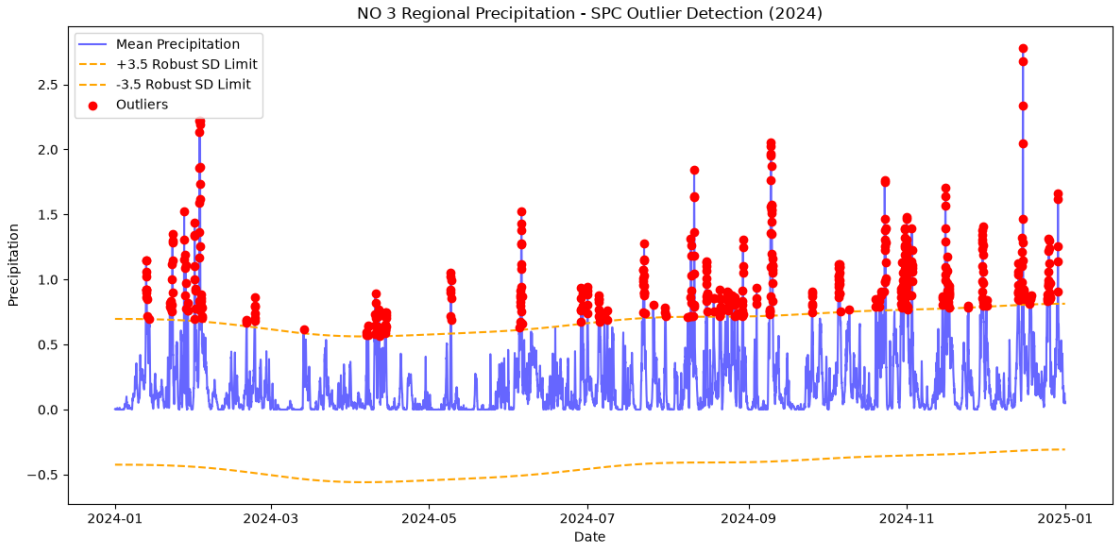
    ax.set_title("NO 3 Regional Precipitation - SPC Outlier Detection (2024)")
    ax.set_xlabel("Date")
    ax.set_ylabel("Precipitation")
    ax.legend()
    plt.tight_layout()

    # Summaries at the bottom of the plot to double check.
    summary = {
        "total_observations": N,
        "outlier_count": outlier_mask.sum(),
        "upper_threshold_satv": upper_limit,
        "lower_threshold_satv": lower_limit,
        "outlier_timestamps": outliers.index.tolist()
    }

    return fig, summary
```

[ ]:
```
# Running the precipitation outlier function.
fig_outliers, summary_outliers = analyze_precipitation_outliers(weather_no3_2024, cutoff_freq=10, num_stds=3.5)
plt.show()
```

```
print("Outlier Analysis Summary:", summary_outliers)
```




```
Outlier Analysis Summary: {'total_observations': 8784, 'outlier_count':
np.int64(491), 'upper_threshold_satv': np.float32(0.47941804),
'lower_threshold_satv': np.float32(-0.64140344), 'outlier_timestamps':
[Timestamp('2024-01-12 21:00:00+0000', tz='UTC'), Timestamp('2024-01-12
22:00:00+0000', tz='UTC'), Timestamp('2024-01-12 23:00:00+0000', tz='UTC'),
Timestamp('2024-01-13 00:00:00+0000', tz='UTC'), Timestamp('2024-01-13
01:00:00+0000', tz='UTC'), Timestamp('2024-01-13 02:00:00+0000', tz='UTC'),
Timestamp('2024-01-13 03:00:00+0000', tz='UTC'), Timestamp('2024-01-13
04:00:00+0000', tz='UTC'), Timestamp('2024-01-13 05:00:00+0000', tz='UTC'),
Timestamp('2024-01-13 06:00:00+0000', tz='UTC'), Timestamp('2024-01-13
07:00:00+0000', tz='UTC'), Timestamp('2024-01-14 04:00:00+0000', tz='UTC'),
Timestamp('2024-01-22 02:00:00+0000', tz='UTC'), Timestamp('2024-01-22
03:00:00+0000', tz='UTC'), Timestamp('2024-01-22 15:00:00+0000', tz='UTC'),
Timestamp('2024-01-22 16:00:00+0000', tz='UTC'), Timestamp('2024-01-22
17:00:00+0000', tz='UTC'), Timestamp('2024-01-22 23:00:00+0000', tz='UTC'),
Timestamp('2024-01-23 00:00:00+0000', tz='UTC'), Timestamp('2024-01-23
01:00:00+0000', tz='UTC'), Timestamp('2024-01-23 02:00:00+0000', tz='UTC'),
Timestamp('2024-01-23 03:00:00+0000', tz='UTC'), Timestamp('2024-01-23
04:00:00+0000', tz='UTC'), Timestamp('2024-01-23 05:00:00+0000', tz='UTC'),
Timestamp('2024-01-23 06:00:00+0000', tz='UTC'), Timestamp('2024-01-27
14:00:00+0000', tz='UTC'), Timestamp('2024-01-27 15:00:00+0000', tz='UTC'),
Timestamp('2024-01-27 16:00:00+0000', tz='UTC'), Timestamp('2024-01-28
01:00:00+0000', tz='UTC'), Timestamp('2024-01-28 02:00:00+0000', tz='UTC'),
Timestamp('2024-01-28 03:00:00+0000', tz='UTC'), Timestamp('2024-01-28
04:00:00+0000', tz='UTC'), Timestamp('2024-01-28 05:00:00+0000', tz='UTC'),
Timestamp('2024-01-28 06:00:00+0000', tz='UTC'), Timestamp('2024-01-29
01:00:00+0000', tz='UTC'), Timestamp('2024-01-29 02:00:00+0000', tz='UTC'),
```

Timestamp('2024-01-29 03:00:00+0000', tz='UTC'), Timestamp('2024-01-31
13:00:00+0000', tz='UTC'), Timestamp('2024-01-31 14:00:00+0000', tz='UTC'),
Timestamp('2024-01-31 15:00:00+0000', tz='UTC'), Timestamp('2024-01-31
16:00:00+0000', tz='UTC'), Timestamp('2024-01-31 17:00:00+0000', tz='UTC'),
Timestamp('2024-01-31 18:00:00+0000', tz='UTC'), Timestamp('2024-01-31
23:00:00+0000', tz='UTC'), Timestamp('2024-02-01 00:00:00+0000', tz='UTC'),
Timestamp('2024-02-01 01:00:00+0000', tz='UTC'), Timestamp('2024-02-02
08:00:00+0000', tz='UTC'), Timestamp('2024-02-02 09:00:00+0000', tz='UTC'),
Timestamp('2024-02-02 10:00:00+0000', tz='UTC'), Timestamp('2024-02-02
11:00:00+0000', tz='UTC'), Timestamp('2024-02-02 12:00:00+0000', tz='UTC'),
Timestamp('2024-02-02 13:00:00+0000', tz='UTC'), Timestamp('2024-02-02
14:00:00+0000', tz='UTC'), Timestamp('2024-02-02 15:00:00+0000', tz='UTC'),
Timestamp('2024-02-02 16:00:00+0000', tz='UTC'), Timestamp('2024-02-02
17:00:00+0000', tz='UTC'), Timestamp('2024-02-02 18:00:00+0000', tz='UTC'),
Timestamp('2024-02-02 19:00:00+0000', tz='UTC'), Timestamp('2024-02-02
20:00:00+0000', tz='UTC'), Timestamp('2024-02-02 21:00:00+0000', tz='UTC'),
Timestamp('2024-02-03 01:00:00+0000', tz='UTC'), Timestamp('2024-02-03
02:00:00+0000', tz='UTC'), Timestamp('2024-02-03 03:00:00+0000', tz='UTC'),
Timestamp('2024-02-03 04:00:00+0000', tz='UTC'), Timestamp('2024-02-03
07:00:00+0000', tz='UTC'), Timestamp('2024-02-03 09:00:00+0000', tz='UTC'),
Timestamp('2024-02-03 10:00:00+0000', tz='UTC'), Timestamp('2024-02-03
11:00:00+0000', tz='UTC'), Timestamp('2024-02-20 16:00:00+0000', tz='UTC'),
Timestamp('2024-02-20 17:00:00+0000', tz='UTC'), Timestamp('2024-02-20
18:00:00+0000', tz='UTC'), Timestamp('2024-02-23 18:00:00+0000', tz='UTC'),
Timestamp('2024-02-23 19:00:00+0000', tz='UTC'), Timestamp('2024-02-23
20:00:00+0000', tz='UTC'), Timestamp('2024-02-23 21:00:00+0000', tz='UTC'),
Timestamp('2024-02-23 22:00:00+0000', tz='UTC'), Timestamp('2024-02-23
23:00:00+0000', tz='UTC'), Timestamp('2024-03-13 15:00:00+0000', tz='UTC'),
Timestamp('2024-04-06 21:00:00+0000', tz='UTC'), Timestamp('2024-04-06
22:00:00+0000', tz='UTC'), Timestamp('2024-04-07 12:00:00+0000', tz='UTC'),
Timestamp('2024-04-07 13:00:00+0000', tz='UTC'), Timestamp('2024-04-07
14:00:00+0000', tz='UTC'), Timestamp('2024-04-10 04:00:00+0000', tz='UTC'),
Timestamp('2024-04-10 05:00:00+0000', tz='UTC'), Timestamp('2024-04-10
06:00:00+0000', tz='UTC'), Timestamp('2024-04-10 07:00:00+0000', tz='UTC'),
Timestamp('2024-04-10 08:00:00+0000', tz='UTC'), Timestamp('2024-04-10
09:00:00+0000', tz='UTC'), Timestamp('2024-04-10 10:00:00+0000', tz='UTC'),
Timestamp('2024-04-11 07:00:00+0000', tz='UTC'), Timestamp('2024-04-11
08:00:00+0000', tz='UTC'), Timestamp('2024-04-11 09:00:00+0000', tz='UTC'),
Timestamp('2024-04-11 14:00:00+0000', tz='UTC'), Timestamp('2024-04-11
15:00:00+0000', tz='UTC'), Timestamp('2024-04-11 16:00:00+0000', tz='UTC'),
Timestamp('2024-04-11 17:00:00+0000', tz='UTC'), Timestamp('2024-04-11
19:00:00+0000', tz='UTC'), Timestamp('2024-04-14 01:00:00+0000', tz='UTC'),
Timestamp('2024-04-14 02:00:00+0000', tz='UTC'), Timestamp('2024-04-14
03:00:00+0000', tz='UTC'), Timestamp('2024-04-14 04:00:00+0000', tz='UTC'),
Timestamp('2024-04-14 05:00:00+0000', tz='UTC'), Timestamp('2024-04-14
06:00:00+0000', tz='UTC'), Timestamp('2024-04-14 07:00:00+0000', tz='UTC'),
Timestamp('2024-04-14 08:00:00+0000', tz='UTC'), Timestamp('2024-04-14
09:00:00+0000', tz='UTC'), Timestamp('2024-04-14 10:00:00+0000', tz='UTC'),

Timestamp('2024-05-09 05:00:00+0000', tz='UTC'), Timestamp('2024-05-09
06:00:00+0000', tz='UTC'), Timestamp('2024-05-09 07:00:00+0000', tz='UTC'),
Timestamp('2024-05-09 08:00:00+0000', tz='UTC'), Timestamp('2024-05-09
09:00:00+0000', tz='UTC'), Timestamp('2024-05-09 10:00:00+0000', tz='UTC'),
Timestamp('2024-05-09 11:00:00+0000', tz='UTC'), Timestamp('2024-05-09
12:00:00+0000', tz='UTC'), Timestamp('2024-06-04 16:00:00+0000', tz='UTC'),
Timestamp('2024-06-05 01:00:00+0000', tz='UTC'), Timestamp('2024-06-05
02:00:00+0000', tz='UTC'), Timestamp('2024-06-05 03:00:00+0000', tz='UTC'),
Timestamp('2024-06-05 04:00:00+0000', tz='UTC'), Timestamp('2024-06-05
05:00:00+0000', tz='UTC'), Timestamp('2024-06-05 06:00:00+0000', tz='UTC'),
Timestamp('2024-06-05 08:00:00+0000', tz='UTC'), Timestamp('2024-06-05
09:00:00+0000', tz='UTC'), Timestamp('2024-06-05 10:00:00+0000', tz='UTC'),
Timestamp('2024-06-05 11:00:00+0000', tz='UTC'), Timestamp('2024-06-05
12:00:00+0000', tz='UTC'), Timestamp('2024-06-05 13:00:00+0000', tz='UTC'),
Timestamp('2024-06-05 14:00:00+0000', tz='UTC'), Timestamp('2024-06-05
15:00:00+0000', tz='UTC'), Timestamp('2024-06-05 16:00:00+0000', tz='UTC'),
Timestamp('2024-06-05 17:00:00+0000', tz='UTC'), Timestamp('2024-06-05
18:00:00+0000', tz='UTC'), Timestamp('2024-06-28 10:00:00+0000', tz='UTC'),
Timestamp('2024-06-28 11:00:00+0000', tz='UTC'), Timestamp('2024-06-28
12:00:00+0000', tz='UTC'), Timestamp('2024-06-28 13:00:00+0000', tz='UTC'),
Timestamp('2024-06-28 14:00:00+0000', tz='UTC'), Timestamp('2024-06-29
04:00:00+0000', tz='UTC'), Timestamp('2024-06-29 05:00:00+0000', tz='UTC'),
Timestamp('2024-06-30 22:00:00+0000', tz='UTC'), Timestamp('2024-06-30
23:00:00+0000', tz='UTC'), Timestamp('2024-07-01 00:00:00+0000', tz='UTC'),
Timestamp('2024-07-01 01:00:00+0000', tz='UTC'), Timestamp('2024-07-01
02:00:00+0000', tz='UTC'), Timestamp('2024-07-05 07:00:00+0000', tz='UTC'),
Timestamp('2024-07-05 08:00:00+0000', tz='UTC'), Timestamp('2024-07-05
09:00:00+0000', tz='UTC'), Timestamp('2024-07-05 10:00:00+0000', tz='UTC'),
Timestamp('2024-07-05 11:00:00+0000', tz='UTC'), Timestamp('2024-07-05
12:00:00+0000', tz='UTC'), Timestamp('2024-07-05 13:00:00+0000', tz='UTC'),
Timestamp('2024-07-05 19:00:00+0000', tz='UTC'), Timestamp('2024-07-07
12:00:00+0000', tz='UTC'), Timestamp('2024-07-08 14:00:00+0000', tz='UTC'),
Timestamp('2024-07-08 16:00:00+0000', tz='UTC'), Timestamp('2024-07-22
09:00:00+0000', tz='UTC'), Timestamp('2024-07-22 10:00:00+0000', tz='UTC'),
Timestamp('2024-07-22 11:00:00+0000', tz='UTC'), Timestamp('2024-07-22
12:00:00+0000', tz='UTC'), Timestamp('2024-07-22 13:00:00+0000', tz='UTC'),
Timestamp('2024-07-22 14:00:00+0000', tz='UTC'), Timestamp('2024-07-22
15:00:00+0000', tz='UTC'), Timestamp('2024-07-22 16:00:00+0000', tz='UTC'),
Timestamp('2024-07-22 17:00:00+0000', tz='UTC'), Timestamp('2024-07-22
18:00:00+0000', tz='UTC'), Timestamp('2024-07-22 19:00:00+0000', tz='UTC'),
Timestamp('2024-07-22 20:00:00+0000', tz='UTC'), Timestamp('2024-07-22
21:00:00+0000', tz='UTC'), Timestamp('2024-07-22 22:00:00+0000', tz='UTC'),
Timestamp('2024-07-22 23:00:00+0000', tz='UTC'), Timestamp('2024-07-23
00:00:00+0000', tz='UTC'), Timestamp('2024-07-26 07:00:00+0000', tz='UTC'),
Timestamp('2024-07-30 22:00:00+0000', tz='UTC'), Timestamp('2024-07-30
23:00:00+0000', tz='UTC'), Timestamp('2024-07-31 01:00:00+0000', tz='UTC'),
Timestamp('2024-07-31 02:00:00+0000', tz='UTC'), Timestamp('2024-08-08
14:00:00+0000', tz='UTC'), Timestamp('2024-08-09 17:00:00+0000', tz='UTC'),

Timestamp('2024-08-09 18:00:00+0000', tz='UTC'), Timestamp('2024-08-09
19:00:00+0000', tz='UTC'), Timestamp('2024-08-09 20:00:00+0000', tz='UTC'),
Timestamp('2024-08-09 21:00:00+0000', tz='UTC'), Timestamp('2024-08-09
22:00:00+0000', tz='UTC'), Timestamp('2024-08-09 23:00:00+0000', tz='UTC'),
Timestamp('2024-08-10 00:00:00+0000', tz='UTC'), Timestamp('2024-08-10
01:00:00+0000', tz='UTC'), Timestamp('2024-08-10 15:00:00+0000', tz='UTC'),
Timestamp('2024-08-10 23:00:00+0000', tz='UTC'), Timestamp('2024-08-11
00:00:00+0000', tz='UTC'), Timestamp('2024-08-11 01:00:00+0000', tz='UTC'),
Timestamp('2024-08-11 02:00:00+0000', tz='UTC'), Timestamp('2024-08-11
03:00:00+0000', tz='UTC'), Timestamp('2024-08-11 04:00:00+0000', tz='UTC'),
Timestamp('2024-08-11 05:00:00+0000', tz='UTC'), Timestamp('2024-08-11
06:00:00+0000', tz='UTC'), Timestamp('2024-08-15 16:00:00+0000', tz='UTC'),
Timestamp('2024-08-15 17:00:00+0000', tz='UTC'), Timestamp('2024-08-15
18:00:00+0000', tz='UTC'), Timestamp('2024-08-15 19:00:00+0000', tz='UTC'),
Timestamp('2024-08-15 20:00:00+0000', tz='UTC'), Timestamp('2024-08-15
21:00:00+0000', tz='UTC'), Timestamp('2024-08-15 22:00:00+0000', tz='UTC'),
Timestamp('2024-08-15 23:00:00+0000', tz='UTC'), Timestamp('2024-08-16
00:00:00+0000', tz='UTC'), Timestamp('2024-08-16 01:00:00+0000', tz='UTC'),
Timestamp('2024-08-16 02:00:00+0000', tz='UTC'), Timestamp('2024-08-16
03:00:00+0000', tz='UTC'), Timestamp('2024-08-18 13:00:00+0000', tz='UTC'),
Timestamp('2024-08-18 14:00:00+0000', tz='UTC'), Timestamp('2024-08-18
15:00:00+0000', tz='UTC'), Timestamp('2024-08-20 21:00:00+0000', tz='UTC'),
Timestamp('2024-08-20 22:00:00+0000', tz='UTC'), Timestamp('2024-08-20
23:00:00+0000', tz='UTC'), Timestamp('2024-08-21 00:00:00+0000', tz='UTC'),
Timestamp('2024-08-21 07:00:00+0000', tz='UTC'), Timestamp('2024-08-21
08:00:00+0000', tz='UTC'), Timestamp('2024-08-21 09:00:00+0000', tz='UTC'),
Timestamp('2024-08-23 19:00:00+0000', tz='UTC'), Timestamp('2024-08-23
20:00:00+0000', tz='UTC'), Timestamp('2024-08-23 21:00:00+0000', tz='UTC'),
Timestamp('2024-08-23 22:00:00+0000', tz='UTC'), Timestamp('2024-08-23
23:00:00+0000', tz='UTC'), Timestamp('2024-08-24 00:00:00+0000', tz='UTC'),
Timestamp('2024-08-24 01:00:00+0000', tz='UTC'), Timestamp('2024-08-24
02:00:00+0000', tz='UTC'), Timestamp('2024-08-25 12:00:00+0000', tz='UTC'),
Timestamp('2024-08-25 13:00:00+0000', tz='UTC'), Timestamp('2024-08-25
14:00:00+0000', tz='UTC'), Timestamp('2024-08-26 12:00:00+0000', tz='UTC'),
Timestamp('2024-08-26 13:00:00+0000', tz='UTC'), Timestamp('2024-08-26
14:00:00+0000', tz='UTC'), Timestamp('2024-08-26 15:00:00+0000', tz='UTC'),
Timestamp('2024-08-26 16:00:00+0000', tz='UTC'), Timestamp('2024-08-27
17:00:00+0000', tz='UTC'), Timestamp('2024-08-27 18:00:00+0000', tz='UTC'),
Timestamp('2024-08-27 19:00:00+0000', tz='UTC'), Timestamp('2024-08-29
15:00:00+0000', tz='UTC'), Timestamp('2024-08-29 16:00:00+0000', tz='UTC'),
Timestamp('2024-08-29 17:00:00+0000', tz='UTC'), Timestamp('2024-08-29
18:00:00+0000', tz='UTC'), Timestamp('2024-08-29 19:00:00+0000', tz='UTC'),
Timestamp('2024-08-29 20:00:00+0000', tz='UTC'), Timestamp('2024-08-29
21:00:00+0000', tz='UTC'), Timestamp('2024-08-29 22:00:00+0000', tz='UTC'),
Timestamp('2024-08-29 23:00:00+0000', tz='UTC'), Timestamp('2024-08-30
00:00:00+0000', tz='UTC'), Timestamp('2024-09-04 02:00:00+0000', tz='UTC'),
Timestamp('2024-09-04 03:00:00+0000', tz='UTC'), Timestamp('2024-09-04
04:00:00+0000', tz='UTC'), Timestamp('2024-09-09 04:00:00+0000', tz='UTC'),

Timestamp('2024-09-09 05:00:00+0000', tz='UTC'), Timestamp('2024-09-09
07:00:00+0000', tz='UTC'), Timestamp('2024-09-09 08:00:00+0000', tz='UTC'),
Timestamp('2024-09-09 09:00:00+0000', tz='UTC'), Timestamp('2024-09-09
10:00:00+0000', tz='UTC'), Timestamp('2024-09-09 11:00:00+0000', tz='UTC'),
Timestamp('2024-09-09 12:00:00+0000', tz='UTC'), Timestamp('2024-09-09
13:00:00+0000', tz='UTC'), Timestamp('2024-09-09 14:00:00+0000', tz='UTC'),
Timestamp('2024-09-09 15:00:00+0000', tz='UTC'), Timestamp('2024-09-09
16:00:00+0000', tz='UTC'), Timestamp('2024-09-09 17:00:00+0000', tz='UTC'),
Timestamp('2024-09-09 18:00:00+0000', tz='UTC'), Timestamp('2024-09-09
19:00:00+0000', tz='UTC'), Timestamp('2024-09-09 20:00:00+0000', tz='UTC'),
Timestamp('2024-09-09 21:00:00+0000', tz='UTC'), Timestamp('2024-09-09
22:00:00+0000', tz='UTC'), Timestamp('2024-09-09 23:00:00+0000', tz='UTC'),
Timestamp('2024-09-10 00:00:00+0000', tz='UTC'), Timestamp('2024-09-10
01:00:00+0000', tz='UTC'), Timestamp('2024-09-10 02:00:00+0000', tz='UTC'),
Timestamp('2024-09-10 03:00:00+0000', tz='UTC'), Timestamp('2024-09-10
04:00:00+0000', tz='UTC'), Timestamp('2024-09-10 05:00:00+0000', tz='UTC'),
Timestamp('2024-09-10 06:00:00+0000', tz='UTC'), Timestamp('2024-09-10
07:00:00+0000', tz='UTC'), Timestamp('2024-09-10 08:00:00+0000', tz='UTC'),
Timestamp('2024-09-10 09:00:00+0000', tz='UTC'), Timestamp('2024-09-25
11:00:00+0000', tz='UTC'), Timestamp('2024-09-25 12:00:00+0000', tz='UTC'),
Timestamp('2024-09-25 13:00:00+0000', tz='UTC'), Timestamp('2024-09-25
14:00:00+0000', tz='UTC'), Timestamp('2024-09-25 15:00:00+0000', tz='UTC'),
Timestamp('2024-09-25 16:00:00+0000', tz='UTC'), Timestamp('2024-10-05
14:00:00+0000', tz='UTC'), Timestamp('2024-10-05 15:00:00+0000', tz='UTC'),
Timestamp('2024-10-05 16:00:00+0000', tz='UTC'), Timestamp('2024-10-05
17:00:00+0000', tz='UTC'), Timestamp('2024-10-05 18:00:00+0000', tz='UTC'),
Timestamp('2024-10-05 19:00:00+0000', tz='UTC'), Timestamp('2024-10-05
20:00:00+0000', tz='UTC'), Timestamp('2024-10-05 21:00:00+0000', tz='UTC'),
Timestamp('2024-10-05 22:00:00+0000', tz='UTC'), Timestamp('2024-10-05
23:00:00+0000', tz='UTC'), Timestamp('2024-10-06 00:00:00+0000', tz='UTC'),
Timestamp('2024-10-06 01:00:00+0000', tz='UTC'), Timestamp('2024-10-06
02:00:00+0000', tz='UTC'), Timestamp('2024-10-06 03:00:00+0000', tz='UTC'),
Timestamp('2024-10-06 04:00:00+0000', tz='UTC'), Timestamp('2024-10-06
05:00:00+0000', tz='UTC'), Timestamp('2024-10-06 06:00:00+0000', tz='UTC'),
Timestamp('2024-10-06 07:00:00+0000', tz='UTC'), Timestamp('2024-10-09
21:00:00+0000', tz='UTC'), Timestamp('2024-10-20 01:00:00+0000', tz='UTC'),
Timestamp('2024-10-20 02:00:00+0000', tz='UTC'), Timestamp('2024-10-20
03:00:00+0000', tz='UTC'), Timestamp('2024-10-21 09:00:00+0000', tz='UTC'),
Timestamp('2024-10-21 12:00:00+0000', tz='UTC'), Timestamp('2024-10-22
04:00:00+0000', tz='UTC'), Timestamp('2024-10-22 05:00:00+0000', tz='UTC'),
Timestamp('2024-10-23 09:00:00+0000', tz='UTC'), Timestamp('2024-10-23
10:00:00+0000', tz='UTC'), Timestamp('2024-10-23 11:00:00+0000', tz='UTC'),
Timestamp('2024-10-23 12:00:00+0000', tz='UTC'), Timestamp('2024-10-23
13:00:00+0000', tz='UTC'), Timestamp('2024-10-23 14:00:00+0000', tz='UTC'),
Timestamp('2024-10-23 15:00:00+0000', tz='UTC'), Timestamp('2024-10-23
16:00:00+0000', tz='UTC'), Timestamp('2024-10-23 17:00:00+0000', tz='UTC'),
Timestamp('2024-10-23 18:00:00+0000', tz='UTC'), Timestamp('2024-10-23
19:00:00+0000', tz='UTC'), Timestamp('2024-10-23 20:00:00+0000', tz='UTC'),

Timestamp('2024-10-23 21:00:00+0000', tz='UTC'), Timestamp('2024-10-23
22:00:00+0000', tz='UTC'), Timestamp('2024-10-29 18:00:00+0000', tz='UTC'),
Timestamp('2024-10-29 19:00:00+0000', tz='UTC'), Timestamp('2024-10-29
20:00:00+0000', tz='UTC'), Timestamp('2024-10-29 21:00:00+0000', tz='UTC'),
Timestamp('2024-10-30 02:00:00+0000', tz='UTC'), Timestamp('2024-10-30
03:00:00+0000', tz='UTC'), Timestamp('2024-10-30 07:00:00+0000', tz='UTC'),
Timestamp('2024-10-30 08:00:00+0000', tz='UTC'), Timestamp('2024-10-30
09:00:00+0000', tz='UTC'), Timestamp('2024-10-30 10:00:00+0000', tz='UTC'),
Timestamp('2024-10-30 11:00:00+0000', tz='UTC'), Timestamp('2024-10-30
12:00:00+0000', tz='UTC'), Timestamp('2024-10-30 13:00:00+0000', tz='UTC'),
Timestamp('2024-10-30 23:00:00+0000', tz='UTC'), Timestamp('2024-10-31
00:00:00+0000', tz='UTC'), Timestamp('2024-10-31 01:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 02:00:00+0000', tz='UTC'), Timestamp('2024-10-31
03:00:00+0000', tz='UTC'), Timestamp('2024-10-31 04:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 05:00:00+0000', tz='UTC'), Timestamp('2024-10-31
06:00:00+0000', tz='UTC'), Timestamp('2024-10-31 07:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 08:00:00+0000', tz='UTC'), Timestamp('2024-10-31
09:00:00+0000', tz='UTC'), Timestamp('2024-10-31 10:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 11:00:00+0000', tz='UTC'), Timestamp('2024-10-31
12:00:00+0000', tz='UTC'), Timestamp('2024-10-31 13:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 14:00:00+0000', tz='UTC'), Timestamp('2024-10-31
15:00:00+0000', tz='UTC'), Timestamp('2024-10-31 16:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 17:00:00+0000', tz='UTC'), Timestamp('2024-10-31
18:00:00+0000', tz='UTC'), Timestamp('2024-10-31 19:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 20:00:00+0000', tz='UTC'), Timestamp('2024-10-31
21:00:00+0000', tz='UTC'), Timestamp('2024-10-31 22:00:00+0000', tz='UTC'),
Timestamp('2024-10-31 23:00:00+0000', tz='UTC'), Timestamp('2024-11-01
00:00:00+0000', tz='UTC'), Timestamp('2024-11-01 01:00:00+0000', tz='UTC'),
Timestamp('2024-11-01 02:00:00+0000', tz='UTC'), Timestamp('2024-11-01
03:00:00+0000', tz='UTC'), Timestamp('2024-11-01 04:00:00+0000', tz='UTC'),
Timestamp('2024-11-01 05:00:00+0000', tz='UTC'), Timestamp('2024-11-01
06:00:00+0000', tz='UTC'), Timestamp('2024-11-01 07:00:00+0000', tz='UTC'),
Timestamp('2024-11-01 10:00:00+0000', tz='UTC'), Timestamp('2024-11-01
11:00:00+0000', tz='UTC'), Timestamp('2024-11-02 20:00:00+0000', tz='UTC'),
Timestamp('2024-11-02 21:00:00+0000', tz='UTC'), Timestamp('2024-11-02
22:00:00+0000', tz='UTC'), Timestamp('2024-11-02 23:00:00+0000', tz='UTC'),
Timestamp('2024-11-03 00:00:00+0000', tz='UTC'), Timestamp('2024-11-03
01:00:00+0000', tz='UTC'), Timestamp('2024-11-03 02:00:00+0000', tz='UTC'),
Timestamp('2024-11-03 03:00:00+0000', tz='UTC'), Timestamp('2024-11-14
16:00:00+0000', tz='UTC'), Timestamp('2024-11-14 17:00:00+0000', tz='UTC'),
Timestamp('2024-11-14 18:00:00+0000', tz='UTC'), Timestamp('2024-11-15
16:00:00+0000', tz='UTC'), Timestamp('2024-11-15 17:00:00+0000', tz='UTC'),
Timestamp('2024-11-15 18:00:00+0000', tz='UTC'), Timestamp('2024-11-15
19:00:00+0000', tz='UTC'), Timestamp('2024-11-15 20:00:00+0000', tz='UTC'),
Timestamp('2024-11-15 21:00:00+0000', tz='UTC'), Timestamp('2024-11-15
22:00:00+0000', tz='UTC'), Timestamp('2024-11-15 23:00:00+0000', tz='UTC'),
Timestamp('2024-11-16 00:00:00+0000', tz='UTC'), Timestamp('2024-11-16
13:00:00+0000', tz='UTC'), Timestamp('2024-11-16 14:00:00+0000', tz='UTC'),

Timestamp('2024-11-16 15:00:00+0000', tz='UTC'), Timestamp('2024-11-16
16:00:00+0000', tz='UTC'), Timestamp('2024-11-16 17:00:00+0000', tz='UTC'),
Timestamp('2024-11-16 18:00:00+0000', tz='UTC'), Timestamp('2024-11-16
19:00:00+0000', tz='UTC'), Timestamp('2024-11-17 00:00:00+0000', tz='UTC'),
Timestamp('2024-11-17 01:00:00+0000', tz='UTC'), Timestamp('2024-11-17
05:00:00+0000', tz='UTC'), Timestamp('2024-11-17 06:00:00+0000', tz='UTC'),
Timestamp('2024-11-17 07:00:00+0000', tz='UTC'), Timestamp('2024-11-17
08:00:00+0000', tz='UTC'), Timestamp('2024-11-17 09:00:00+0000', tz='UTC'),
Timestamp('2024-11-17 10:00:00+0000', tz='UTC'), Timestamp('2024-11-17
11:00:00+0000', tz='UTC'), Timestamp('2024-11-24 17:00:00+0000', tz='UTC'),
Timestamp('2024-11-24 18:00:00+0000', tz='UTC'), Timestamp('2024-11-29
21:00:00+0000', tz='UTC'), Timestamp('2024-11-29 22:00:00+0000', tz='UTC'),
Timestamp('2024-11-29 23:00:00+0000', tz='UTC'), Timestamp('2024-11-30
00:00:00+0000', tz='UTC'), Timestamp('2024-11-30 01:00:00+0000', tz='UTC'),
Timestamp('2024-11-30 02:00:00+0000', tz='UTC'), Timestamp('2024-11-30
03:00:00+0000', tz='UTC'), Timestamp('2024-11-30 04:00:00+0000', tz='UTC'),
Timestamp('2024-11-30 05:00:00+0000', tz='UTC'), Timestamp('2024-11-30
06:00:00+0000', tz='UTC'), Timestamp('2024-11-30 07:00:00+0000', tz='UTC'),
Timestamp('2024-11-30 08:00:00+0000', tz='UTC'), Timestamp('2024-11-30
09:00:00+0000', tz='UTC'), Timestamp('2024-11-30 10:00:00+0000', tz='UTC'),
Timestamp('2024-11-30 11:00:00+0000', tz='UTC'), Timestamp('2024-11-30
12:00:00+0000', tz='UTC'), Timestamp('2024-12-01 16:00:00+0000', tz='UTC'),
Timestamp('2024-12-01 17:00:00+0000', tz='UTC'), Timestamp('2024-12-01
18:00:00+0000', tz='UTC'), Timestamp('2024-12-13 18:00:00+0000', tz='UTC'),
Timestamp('2024-12-13 19:00:00+0000', tz='UTC'), Timestamp('2024-12-13
20:00:00+0000', tz='UTC'), Timestamp('2024-12-13 21:00:00+0000', tz='UTC'),
Timestamp('2024-12-13 22:00:00+0000', tz='UTC'), Timestamp('2024-12-13
23:00:00+0000', tz='UTC'), Timestamp('2024-12-14 00:00:00+0000', tz='UTC'),
Timestamp('2024-12-14 01:00:00+0000', tz='UTC'), Timestamp('2024-12-14
02:00:00+0000', tz='UTC'), Timestamp('2024-12-14 03:00:00+0000', tz='UTC'),
Timestamp('2024-12-15 08:00:00+0000', tz='UTC'), Timestamp('2024-12-15
09:00:00+0000', tz='UTC'), Timestamp('2024-12-15 10:00:00+0000', tz='UTC'),
Timestamp('2024-12-15 11:00:00+0000', tz='UTC'), Timestamp('2024-12-15
12:00:00+0000', tz='UTC'), Timestamp('2024-12-15 13:00:00+0000', tz='UTC'),
Timestamp('2024-12-15 14:00:00+0000', tz='UTC'), Timestamp('2024-12-15
15:00:00+0000', tz='UTC'), Timestamp('2024-12-15 16:00:00+0000', tz='UTC'),
Timestamp('2024-12-15 17:00:00+0000', tz='UTC'), Timestamp('2024-12-15
18:00:00+0000', tz='UTC'), Timestamp('2024-12-15 19:00:00+0000', tz='UTC'),
Timestamp('2024-12-15 20:00:00+0000', tz='UTC'), Timestamp('2024-12-15
21:00:00+0000', tz='UTC'), Timestamp('2024-12-15 22:00:00+0000', tz='UTC'),
Timestamp('2024-12-15 23:00:00+0000', tz='UTC'), Timestamp('2024-12-16
00:00:00+0000', tz='UTC'), Timestamp('2024-12-18 10:00:00+0000', tz='UTC'),
Timestamp('2024-12-18 11:00:00+0000', tz='UTC'), Timestamp('2024-12-18
12:00:00+0000', tz='UTC'), Timestamp('2024-12-18 13:00:00+0000', tz='UTC'),
Timestamp('2024-12-18 19:00:00+0000', tz='UTC'), Timestamp('2024-12-25
00:00:00+0000', tz='UTC'), Timestamp('2024-12-25 01:00:00+0000', tz='UTC'),
Timestamp('2024-12-25 02:00:00+0000', tz='UTC'), Timestamp('2024-12-25
03:00:00+0000', tz='UTC'), Timestamp('2024-12-25 08:00:00+0000', tz='UTC'),

```
Timestamp('2024-12-25 09:00:00+0000', tz='UTC'), Timestamp('2024-12-25
10:00:00+0000', tz='UTC'), Timestamp('2024-12-25 11:00:00+0000', tz='UTC'),
Timestamp('2024-12-25 12:00:00+0000', tz='UTC'), Timestamp('2024-12-25
13:00:00+0000', tz='UTC'), Timestamp('2024-12-25 14:00:00+0000', tz='UTC'),
Timestamp('2024-12-25 15:00:00+0000', tz='UTC'), Timestamp('2024-12-25
16:00:00+0000', tz='UTC'), Timestamp('2024-12-25 17:00:00+0000', tz='UTC'),
Timestamp('2024-12-25 18:00:00+0000', tz='UTC'), Timestamp('2024-12-25
19:00:00+0000', tz='UTC'), Timestamp('2024-12-25 20:00:00+0000', tz='UTC'),
Timestamp('2024-12-25 23:00:00+0000', tz='UTC'), Timestamp('2024-12-26
00:00:00+0000', tz='UTC'), Timestamp('2024-12-26 01:00:00+0000', tz='UTC'),
Timestamp('2024-12-26 02:00:00+0000', tz='UTC'), Timestamp('2024-12-26
03:00:00+0000', tz='UTC'), Timestamp('2024-12-26 04:00:00+0000', tz='UTC'),
Timestamp('2024-12-29 04:00:00+0000', tz='UTC'), Timestamp('2024-12-29
05:00:00+0000', tz='UTC'), Timestamp('2024-12-29 06:00:00+0000', tz='UTC'),
Timestamp('2024-12-29 07:00:00+0000', tz='UTC'), Timestamp('2024-12-29
08:00:00+0000', tz='UTC')]}
```

1.8 LOF function

[ ]:
```python
def analyze_temperature_anomalies_lof(df_weather, contamination=0.01):
    """
    This function analyzes regional average temperature for anomalies using the
    Local Outlier Factor (LOF) method.
    """
    # Aggregate to average temperature across all points over time.
    df_sorted = df_weather.copy()
    df_sorted['date'] = pd.to_datetime(df_sorted['date'])

    time_series = df_sorted.groupby('date')['temperature_2m'].mean().
↪reset_index()
    time_series = time_series.sort_values('date').set_index('date')

    temperatures = time_series['temperature_2m'].values

    # Reshaping input for scikit-learn model and fit LOF.
    X = temperatures.reshape(-1, 1)
    lof = LocalOutlierFactor(n_neighbors=20, contamination=contamination)
    preds = lof.fit_predict(X)  # -1 indicates an outlier, 1 indicates normal␣
↪data.

    time_series['is_outlier'] = (preds == -1)

    # Plotting the anomalies.
    fig, ax = plt.subplots(figsize=(12, 6))

    ax.plot(time_series.index, time_series['temperature_2m'], color='tab:blue',␣
↪alpha=0.7, label='Mean Temperature')
```

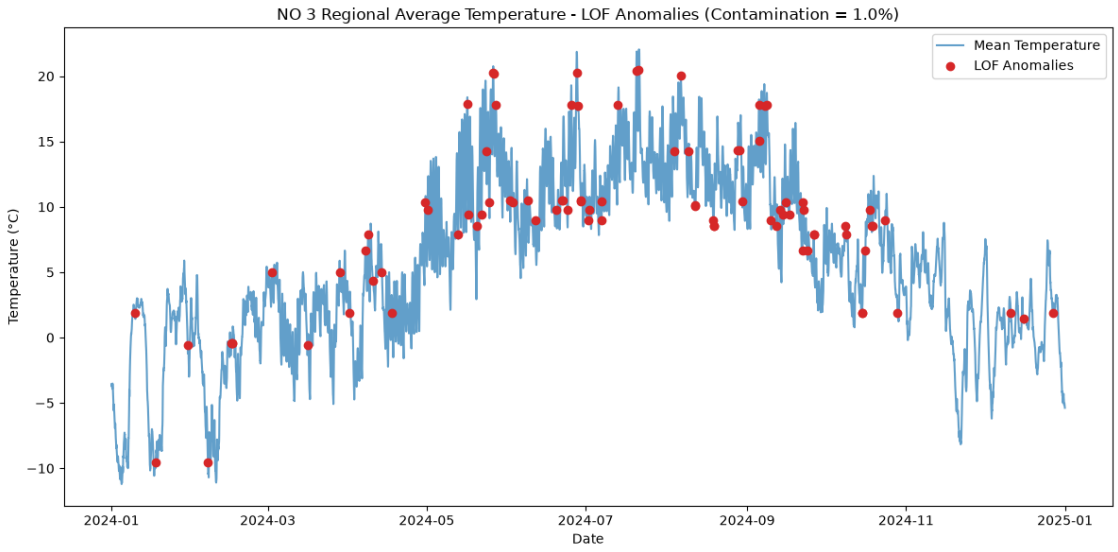
```
    outliers = time_series[time_series['is_outlier']]
    ax.scatter(outliers.index, outliers['temperature_2m'], color='tab:red',␣
↪zorder=5, label='LOF Anomalies')

    ax.set_title(f"NO 3 Regional Average Temperature - LOF Anomalies␣
↪(Contamination = {contamination*100}%)")
    ax.set_xlabel("Date")
    ax.set_ylabel("Temperature (°C)")
    ax.legend()
    plt.tight_layout()

    # Summaries at the bottom of the plot to double check.
    summary = {
        "total_observations": len(time_series),
        "outlier_count": int(time_series['is_outlier'].sum()),
        "contamination_setting": contamination,
        "anomaly_timestamps": outliers.index.tolist()
    }

    return fig, summary
```

[ ]:
```
# Running the LOF anomaly function.
fig_temp_lof, summary_temp_lof =␣
↪analyze_temperature_anomalies_lof(weather_no3_2024, contamination=0.01)
plt.show()
print("Temperature LOF Summary:", summary_temp_lof)
```




```
Temperature LOF Summary: {'total_observations': 8784, 'outlier_count': 88,
```

'contamination_setting': 0.01, 'anomaly_timestamps': [Timestamp('2024-01-09
23:00:00+0000', tz='UTC'), Timestamp('2024-01-17 22:00:00+0000', tz='UTC'),
Timestamp('2024-01-30 11:00:00+0000', tz='UTC'), Timestamp('2024-02-07
00:00:00+0000', tz='UTC'), Timestamp('2024-02-16 03:00:00+0000', tz='UTC'),
Timestamp('2024-02-16 05:00:00+0000', tz='UTC'), Timestamp('2024-02-16
06:00:00+0000', tz='UTC'), Timestamp('2024-03-02 12:00:00+0000', tz='UTC'),
Timestamp('2024-03-16 09:00:00+0000', tz='UTC'), Timestamp('2024-03-28
17:00:00+0000', tz='UTC'), Timestamp('2024-04-01 04:00:00+0000', tz='UTC'),
Timestamp('2024-04-07 12:00:00+0000', tz='UTC'), Timestamp('2024-04-08
12:00:00+0000', tz='UTC'), Timestamp('2024-04-10 12:00:00+0000', tz='UTC'),
Timestamp('2024-04-13 19:00:00+0000', tz='UTC'), Timestamp('2024-04-17
18:00:00+0000', tz='UTC'), Timestamp('2024-04-30 12:00:00+0000', tz='UTC'),
Timestamp('2024-05-01 08:00:00+0000', tz='UTC'), Timestamp('2024-05-13
04:00:00+0000', tz='UTC'), Timestamp('2024-05-16 12:00:00+0000', tz='UTC'),
Timestamp('2024-05-17 03:00:00+0000', tz='UTC'), Timestamp('2024-05-20
08:00:00+0000', tz='UTC'), Timestamp('2024-05-22 02:00:00+0000', tz='UTC'),
Timestamp('2024-05-23 21:00:00+0000', tz='UTC'), Timestamp('2024-05-24
23:00:00+0000', tz='UTC'), Timestamp('2024-05-26 11:00:00+0000', tz='UTC'),
Timestamp('2024-05-26 15:00:00+0000', tz='UTC'), Timestamp('2024-05-27
14:00:00+0000', tz='UTC'), Timestamp('2024-06-01 21:00:00+0000', tz='UTC'),
Timestamp('2024-06-03 01:00:00+0000', tz='UTC'), Timestamp('2024-06-08
17:00:00+0000', tz='UTC'), Timestamp('2024-06-11 15:00:00+0000', tz='UTC'),
Timestamp('2024-06-19 16:00:00+0000', tz='UTC'), Timestamp('2024-06-22
00:00:00+0000', tz='UTC'), Timestamp('2024-06-22 03:00:00+0000', tz='UTC'),
Timestamp('2024-06-24 05:00:00+0000', tz='UTC'), Timestamp('2024-06-25
15:00:00+0000', tz='UTC'), Timestamp('2024-06-27 19:00:00+0000', tz='UTC'),
Timestamp('2024-06-28 00:00:00+0000', tz='UTC'), Timestamp('2024-06-29
05:00:00+0000', tz='UTC'), Timestamp('2024-06-29 07:00:00+0000', tz='UTC'),
Timestamp('2024-07-02 05:00:00+0000', tz='UTC'), Timestamp('2024-07-02
08:00:00+0000', tz='UTC'), Timestamp('2024-07-07 00:00:00+0000', tz='UTC'),
Timestamp('2024-07-07 06:00:00+0000', tz='UTC'), Timestamp('2024-07-13
11:00:00+0000', tz='UTC'), Timestamp('2024-07-20 11:00:00+0000', tz='UTC'),
Timestamp('2024-07-21 08:00:00+0000', tz='UTC'), Timestamp('2024-08-03
22:00:00+0000', tz='UTC'), Timestamp('2024-08-06 14:00:00+0000', tz='UTC'),
Timestamp('2024-08-09 10:00:00+0000', tz='UTC'), Timestamp('2024-08-11
23:00:00+0000', tz='UTC'), Timestamp('2024-08-12 02:00:00+0000', tz='UTC'),
Timestamp('2024-08-18 23:00:00+0000', tz='UTC'), Timestamp('2024-08-19
03:00:00+0000', tz='UTC'), Timestamp('2024-08-19 04:00:00+0000', tz='UTC'),
Timestamp('2024-08-28 09:00:00+0000', tz='UTC'), Timestamp('2024-08-29
03:00:00+0000', tz='UTC'), Timestamp('2024-08-30 06:00:00+0000', tz='UTC'),
Timestamp('2024-09-05 16:00:00+0000', tz='UTC'), Timestamp('2024-09-05
19:00:00+0000', tz='UTC'), Timestamp('2024-09-07 17:00:00+0000', tz='UTC'),
Timestamp('2024-09-08 10:00:00+0000', tz='UTC'), Timestamp('2024-09-10
03:00:00+0000', tz='UTC'), Timestamp('2024-09-12 04:00:00+0000', tz='UTC'),
Timestamp('2024-09-13 13:00:00+0000', tz='UTC'), Timestamp('2024-09-14
18:00:00+0000', tz='UTC'), Timestamp('2024-09-16 00:00:00+0000', tz='UTC'),
Timestamp('2024-09-17 03:00:00+0000', tz='UTC'), Timestamp('2024-09-22
04:00:00+0000', tz='UTC'), Timestamp('2024-09-22 12:00:00+0000', tz='UTC'),

```
Timestamp('2024-09-22 16:00:00+0000', tz='UTC'), Timestamp('2024-09-24
08:00:00+0000', tz='UTC'), Timestamp('2024-09-26 13:00:00+0000', tz='UTC'),
Timestamp('2024-09-26 14:00:00+0000', tz='UTC'), Timestamp('2024-10-08
13:00:00+0000', tz='UTC'), Timestamp('2024-10-08 22:00:00+0000', tz='UTC'),
Timestamp('2024-10-15 02:00:00+0000', tz='UTC'), Timestamp('2024-10-15
03:00:00+0000', tz='UTC'), Timestamp('2024-10-16 10:00:00+0000', tz='UTC'),
Timestamp('2024-10-17 23:00:00+0000', tz='UTC'), Timestamp('2024-10-18
19:00:00+0000', tz='UTC'), Timestamp('2024-10-19 00:00:00+0000', tz='UTC'),
Timestamp('2024-10-23 22:00:00+0000', tz='UTC'), Timestamp('2024-10-28
18:00:00+0000', tz='UTC'), Timestamp('2024-12-11 05:00:00+0000', tz='UTC'),
Timestamp('2024-12-16 00:00:00+0000', tz='UTC'), Timestamp('2024-12-27
09:00:00+0000', tz='UTC')]}
```

[ ]:
```
# Downloading the magasinstatistikk again.
# The code below is copied from previous assignment.
url = "https://biapi.nve.no/magasinstatistikk/api/Magasinstatistikk/
 ↪HentOffentligData"
response = requests.get(url)
data = response.json()

reservoir_df = pd.DataFrame(data)
```

[21]:
```
# Renaming the columns.
reservoir_df = reservoir_df.rename(columns={"dato_Id": "date_Id", "omrType":␣
 ↪"areaType", "omrnr": "areaNr", "iso_aar": "iso_year",
                                             "iso_uke": "iso_week",␣
 ↪"fyllingsgrad": "fill_level", "kapasitet_TWh": "capacity_TWh",
                                             "fylling_TWh": "fill_TWh",␣
 ↪"neste_Publiseringsdato": "next_publishdate",
                                             "fyllingsgrad_forrige_uke":␣
 ↪"fill_level_lastweek", "endring_fyllingsgrad": "fill_level_change"})
```

[22]:
```
# Converting the "date_Id" column to datetime.
reservoir_df['date_Id'] = pd.to_datetime(reservoir_df['date_Id'])

# Sorting the dataframe according to date.
reservoir_df = reservoir_df.sort_values(by='date_Id').reset_index(drop=True)
display(reservoir_df)
```

```
         date_Id areaType  areaNr  iso_year  iso_week  fill_level  \
0     1995-01-08       EL       5      1995         1    0.616448
1     1995-01-08     VASS       1      1995         1    0.635440
2     1995-01-08       EL       2      1995         1    0.630947
3     1995-01-08       EL       3      1995         1    0.602376
4     1995-01-08       EL       1      1995         1    0.617574
…            …      …      …         …         …          …
14944 2026-11-01       EL       3      2026        44    0.720149
14945 2026-11-01       EL       4      2026        44    0.905202
```

```
14946 2026-11-01     VASS       2      2026        44    0.613036
14947 2026-11-01     VASS       3      2026        44    0.850322
14948 2026-11-01       EL       1      2026        44    0.815520

       capacity_TWh   fill_TWh     next_publishdate  fill_level_lastweek  \
0         17.390587  10.720388  0001-01-01T00:00:00             0.614288
1         36.044464  22.904102  0001-01-01T00:00:00             0.756482
2         34.043590  21.479713  0001-01-01T00:00:00             0.777604
3          8.920932   5.373754  0001-01-01T00:00:00             0.596027
4          6.003264   3.707457  0001-01-01T00:00:00             0.583705
…               …          …                    …                    …
14944      8.920932   6.424399  2026-11-11T13:00:00             0.717935
14945     21.079208  19.080936  2026-11-11T13:00:00             0.904775
14946     23.237715  14.245566  2026-11-11T13:00:00             0.601080
14947     28.155403  23.941158  2026-11-11T13:00:00             0.850157
14948      6.003264   4.895780  2026-11-11T13:00:00             0.778386

       fill_level_change
0               0.002160
1              -0.121041
2              -0.146657
3               0.006349
4               0.033869
…                    …
14944           0.002214
14945           0.000427
14946           0.011956
14947           0.000165
14948           0.037134

[14949 rows x 11 columns]
```

1.9 STL Decomposition

```
[ ]: def analyze_reservoir_stl(reservoir_df, area_type='EL', area_nr=3,
                               seasonal=13, trend=None, robust=True, period=52):
         """
         This function performs STL decomposition on the amount of filling for a␣
      ↪specific area.
         """
         # Filtering for NO3.
         # Making sure types and numbers match together.
         df_filtered = reservoir_df[(reservoir_df['areaType'] == area_type) &␣
      ↪(reservoir_df['areaNr'].astype(str) == str(area_nr))].copy()

         # Sorting according to timestamp.
         df_filtered = df_filtered.sort_values('date_Id').set_index('date_Id')
```

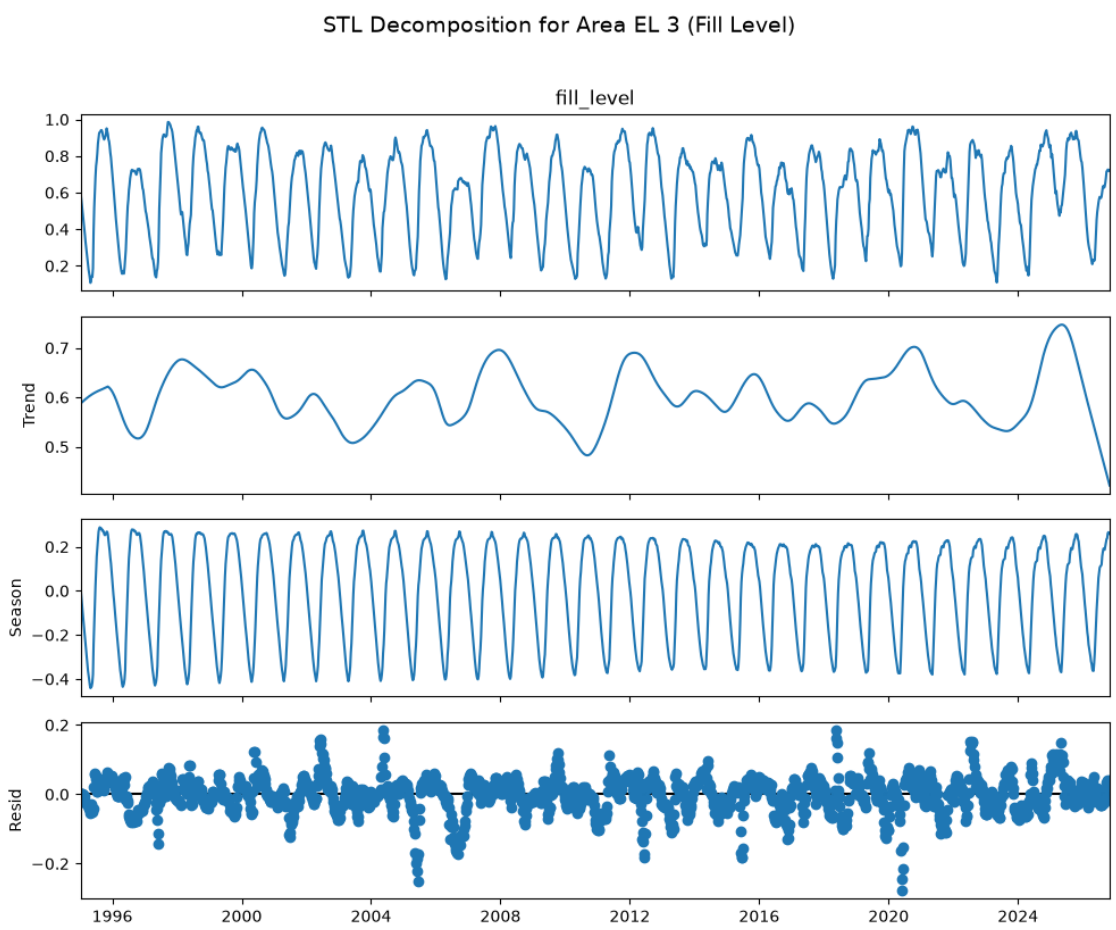
```
    # Dropping null values just in case.
    fill_series = df_filtered['fill_level'].dropna()

    # Fitting STL.
    stl = STL(fill_series, period=period, seasonal=seasonal, trend=trend,␣
 ↪robust=robust)
    result = stl.fit()

    # Plotting the STL.
    fig = result.plot()
    fig.set_size_inches(10, 8)
    plt.suptitle(f"STL Decomposition for Area {area_type} {area_nr} (Fill␣
 ↪Level)", fontsize=14, y=1.02)
    plt.tight_layout()

    return fig, result
```

[ ]:
```
# Putting everything together.
fig_stl, stl_result = analyze_reservoir_stl(reservoir_df, area_type='EL',␣
 ↪area_nr=3,
                                            seasonal=13, trend=None,␣
 ↪robust=True)
plt.show()
```



1.10 Spectogram

[ ]:
```
def plot_reservoir_spectrogram(reservoir_df, area_type='EL', area_nr=3,
                               nperseg=104, noverlap=52, fs=52):
    """
    This function creates and returns a spectrogram plot for the reservoir fill␣
↪level
    of a specific area (in this case NO3) using scipy spectrogram function.
    """
    # Filter for the specified area.
    df_filtered = reservoir_df[(reservoir_df['areaType'] == area_type) &␣
↪(reservoir_df['areaNr'].astype(str) == str(area_nr))].copy()

    df_filtered = df_filtered.sort_values('date_Id').set_index('date_Id')
    fill_series = df_filtered['fill_level'].dropna()

    # Detrending for focusing purposes.
    signal = detrend(fill_series.values)
```

```
    # Computes the spectrogram using scipy.
    f, t, Sxx = spectrogram(signal, fs=fs, nperseg=nperseg, noverlap=noverlap)

    # Converting the time array (from years) to actual dates for plotting.
    start_date = fill_series.index[0]
    dates = start_date + pd.to_timedelta(t * 365.25, unit='D')

    # Plotting the spectogram.
    fig, ax = plt.subplots(figsize=(12, 6))

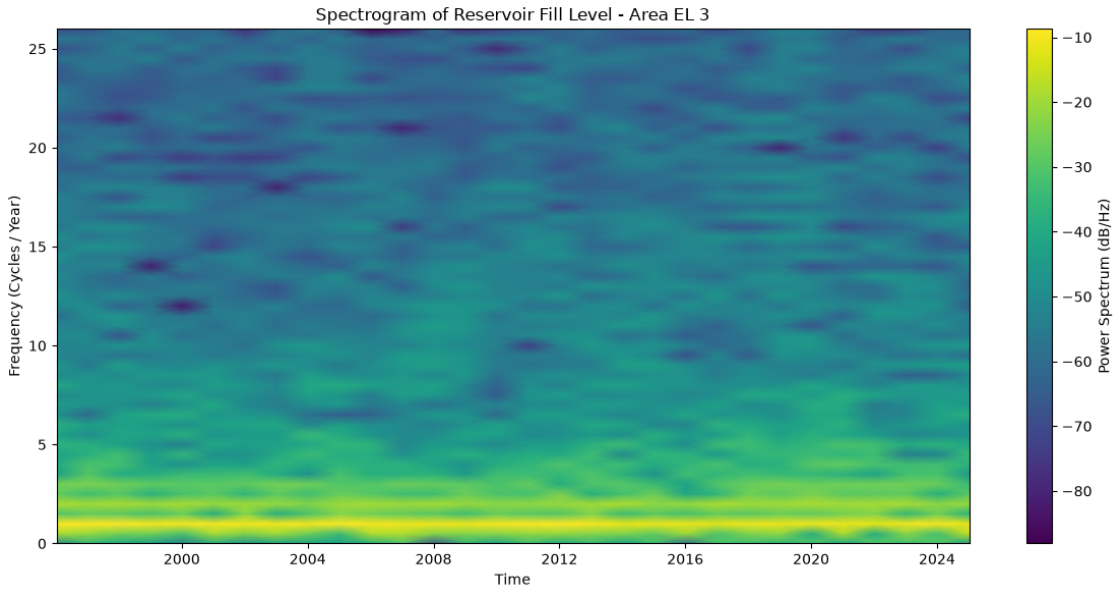
    # It was messy so I looked it up and decided to use dB.
    Sxx_db = 10 * np.log10(Sxx + 1e-10)
    im = ax.pcolormesh(dates, f, Sxx_db, shading='gouraud', cmap='viridis')

    ax.set_ylabel('Frequency (Cycles / Year)')
    ax.set_xlabel('Time')
    ax.set_title(f'Spectrogram of Reservoir Fill Level - Area {area_type}␣
 ↪{area_nr}')

    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label('Power Spectrum (dB/Hz)')
    plt.tight_layout()

    return fig
```

[ ]:
```
# Plotting the spectogram.
fig_spectrogram = plot_reservoir_spectrogram(reservoir_df, area_type='EL',␣
 ↪area_nr=3,
                                             nperseg=104, noverlap=52, fs=52)
plt.show()
```


Spectrogram of Reservoir Fill Level - Area EL 3
Frequency (Cycles / Year)
Time
Power Spectrum (dB/Hz)
0
5
10
15
20
25
2000
2004
2008
2012
2016
2020
2024
−10
−20
−30
−40
−50
−60
−70
−80